CS 301
High-Performance Computing

Assignment 04 - Particle Interpolation and Parallel Mover Performance

Dhruvi Jobanputra (202301401)
Prasham Shah (202301252)

February 2026

Contents

1 Introduction **3**

2 Problem Description **3**

3 Implementation Approach **3**
- 3.1 Cloud-In-Cell Interpolation . . . 4
- 3.2 Serial Particle Mover . . . 4
- 3.3 OpenMP Parallel Mover . . . 5

4 Experimental Setup **5**
- 4.1 Systems Description . . . 5
- 4.2 Experiment Definitions . . . 5
  - 4.2.1 Experiment 1: Scaling with Particle Count . . . 6
  - 4.2.2 Experiment 2: Consistency Across Configurations . . . 6
  - 4.2.3 Experiment 3: Mover Parallelization . . . 6

5 Results **7**
- 5.1 Experiment 1: Scaling with Particle Count . . . 7
- 5.2 Experiment 2: Consistency Across Configurations . . . 8
- 5.3 Experiment 3: Serial vs Parallel Mover . . . 10
- 5.4 Lab vs HPC Comparison . . . 11

6 Analysis Tasks **11**

7 Conclusion **13**

1 Introduction

Particle–mesh methods are widely used in scientific simulations such as plasma physics, molecular dynamics, and fluid modeling. These methods represent physical quantities using particles while solving governing equations on a structured grid. A fundamental operation in such simulations is mapping particle information onto the grid using interpolation schemes.

In this assignment, the Cloud-In-Cell (CIC) bilinear interpolation algorithm is implemented to distribute particle contributions onto a structured mesh. In addition to interpolation, a stochastic particle mover step is introduced to update particle positions over time.

The performance of the interpolation and mover operations is analyzed across multiple grid configurations and particle counts on both a laboratory workstation and an HPC cluster. The mover operation is further parallelized using OpenMP to evaluate multicore scalability and compare serial and parallel execution performance.

2 Problem Description

Consider a unit square domain containing randomly distributed particles with coordinates (x_i, y_i). The domain is discretized into a structured grid of size $N_X \times N_Y$. Each particle lies inside a grid cell bounded by four neighboring grid nodes.

The objective is to perform the following operations:

- Interpolate particle contributions to the four surrounding grid nodes using the Cloud-In-Cell (CIC) bilinear interpolation scheme.
- Update particle positions through a stochastic mover that displaces particles within local grid spacing while enforcing domain boundaries.
- Evaluate performance scaling with particle count and grid size.
- Parallelize the mover using OpenMP and measure speedup.
- Compare performance on a Lab PC and an HPC cluster.

Three experiments are conducted:

- **Experiment 1:** Interpolation scaling with particle count.
- **Experiment 2:** Consistency across grid configurations.
- **Experiment 3:** Serial vs OpenMP mover performance.

3 Implementation Approach

The particle–mesh workflow consists of interpolation followed by particle movement. Efficient data layout, indexing, and loop organization are critical for achieving good cache utilization and computational performance.

3.1 Cloud-In-Cell Interpolation

The structured grid is stored in a one-dimensional array in row-major order. A grid node at logical index (i, j) is accessed as:

$$\texttt{mesh}[j \cdot GRID_X + i]$$

Each particle located inside cell (i, j) distributes its weight to the four surrounding grid nodes. Local coordinates inside the cell are:

$$l_x = x - i\Delta x, \quad l_y = y - j\Delta y$$

The bilinear CIC weights are:

$$\begin{aligned}
w_{00} &= (1 - l_x/\Delta x)(1 - l_y/\Delta y) \\
w_{10} &= (l_x/\Delta x)(1 - l_y/\Delta y) \\
w_{01} &= (1 - l_x/\Delta x)(l_y/\Delta y) \\
w_{11} &= (l_x/\Delta x)(l_y/\Delta y)
\end{aligned}$$

Each particle performs four grid updates:

$$F_{i,j} += w_{00}, \quad F_{i+1,j} += w_{10}, \quad F_{i,j+1} += w_{01}, \quad F_{i+1,j+1} += w_{11}$$

Since the operations per particle are constant, interpolation complexity scales linearly with particle count.

3.2 Serial Particle Mover

The stochastic mover updates particle positions by adding random displacements bounded by grid spacing:

$$x' = x + \delta_x, \quad y' = y + \delta_y$$

where

$$\delta_x \in [-\Delta x, \Delta x], \quad \delta_y \in [-\Delta y, \Delta y]$$

A rejection condition ensures particles remain inside the domain:

$$0 \le x' \le 1, \quad 0 \le y' \le 1$$

Each particle is updated independently, making the mover computationally intensive but embarrassingly parallel.

3.3 OpenMP Parallel Mover

The mover loop is parallelized using OpenMP with static scheduling. Each thread processes a distinct subset of particles and uses a thread-local random seed to avoid contention:

```
#pragma omp parallel for schedule(static)
for (long long p = 0; p < NUM_Points; p++) {
    ...
}
```

Because particle updates are independent and no shared grid writes occur, the parallel mover achieves high scalability and significant runtime reduction.

4 Experimental Setup

All experiments were conducted using the same optimized particle–mesh code on two different computing environments: a laboratory workstation and an HPC cluster node. Execution time was measured using high-resolution wall-clock timing for interpolation and mover phases separately.

4.1 Systems Description

Lab PC:

- Desktop workstation with multi-core CPU
- Higher single-core clock frequency
- Lower memory latency
- OpenMP threads: 4

HPC Cluster Node:

- Multi-core server processor
- Larger shared memory hierarchy
- Higher memory bandwidth but lower per-core frequency
- OpenMP threads: 4 (single node execution)

The code was compiled with `-O3` optimization and OpenMP support on both systems to ensure comparable execution conditions.

4.2 Experiment Definitions

Three experiments were designed to evaluate interpolation scaling, configuration dependence, and parallel mover performance.

4.2.1 Experiment 1: Scaling with Particle Count

Interpolation runtime was measured for increasing particle counts while varying grid resolution. Particles were reinitialized at each iteration to avoid spatial correlation.

Grid Configurations	Particle Counts
250×100	$10^2,\ 10^4,\ 10^6,\ 10^8$
500×200	$10^2,\ 10^4,\ 10^6,\ 10^8$
1000×400	$10^2,\ 10^4,\ 10^6,\ 10^8$

Each case was executed for 10 iterations and total interpolation time was recorded.

4.2.2 Experiment 2: Consistency Across Configurations

Interpolation performance was evaluated across grid resolutions while keeping particle count fixed at 10^8. Particles were initialized once and reused across iterations to ensure consistent spatial distribution.

Problem Index	Grid Size
1	250×100
2	500×200
3	1000×400

Total interpolation time over 10 iterations was measured.

4.2.3 Experiment 3: Mover Parallelization

The stochastic mover was evaluated on a fixed configuration:

$$N_X = 1000, \quad N_Y = 400, \quad N_p = 14\,\text{million}$$

Serial and OpenMP parallel mover implementations were executed for 10 iterations. Average mover runtime and speedup were computed as:

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

This experiment evaluates multicore scalability and the effectiveness of OpenMP parallelization.

5 Results

Execution time was measured for interpolation scaling, configuration consistency, and mover parallelization on both Lab PC and HPC systems.

5.1 Experiment 1: Scaling with Particle Count

Total interpolation time increased approximately linearly with particle count for all grid configurations on both systems, confirming $O(N)$ scaling behavior.

Grid	10^2	10^4	10^6	10^8
250×100 (Lab)	8e-6	4.62e-4	4.25e-2	4.32
500×200 (Lab)	7e-6	4.31e-4	4.38e-2	4.51
1000×400 (Lab)	1e-5	7.52e-4	7.53e-2	7.65
250×100 (HPC)	2.1e-5	2.40e-3	0.2339	23.49
500×200 (HPC)	3.6e-5	2.57e-3	0.2518	20.63
1000×400 (HPC)	6.0e-5	4.73e-3	0.2731	20.78

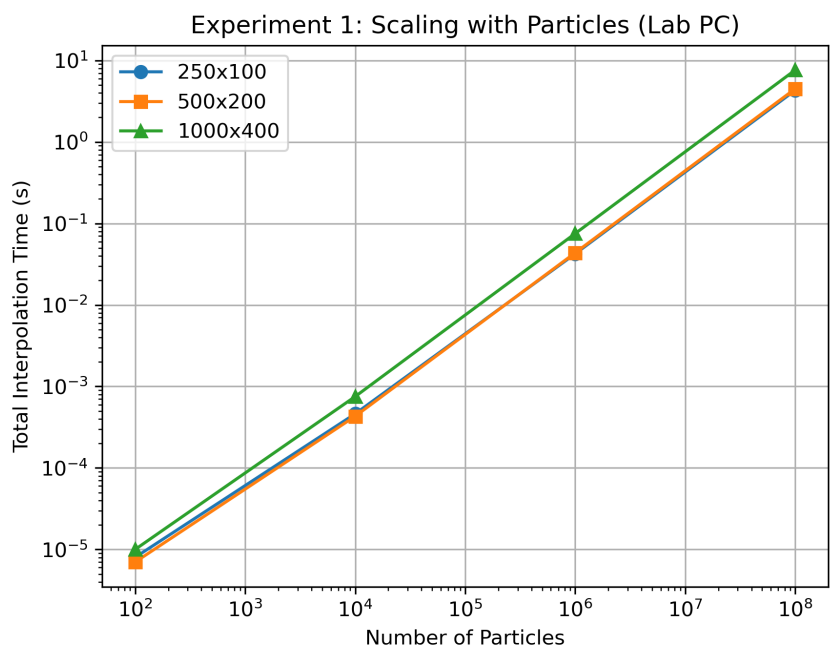

Figure 1: Interpolation scaling with particle count on Lab PC.

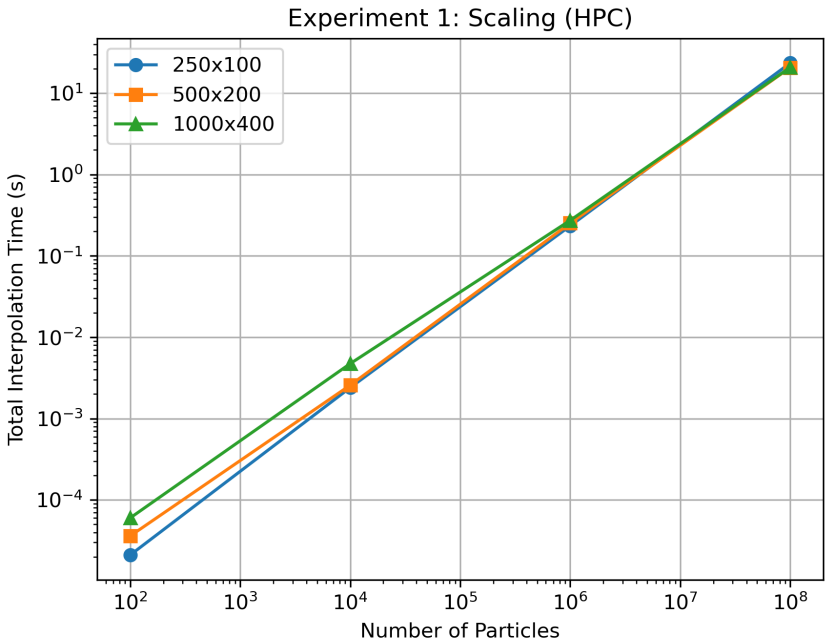

Figure 2: Interpolation scaling with particle count on HPC cluster.

5.2 Experiment 2: Consistency Across Configurations

Interpolation runtime increased moderately with grid size when particle count was fixed, reflecting increased memory footprint and cache pressure.

Grid	Lab PC (s)	HPC (s)
250×100	4.3246	21.6401
500×200	4.4521	24.5539
1000×400	7.3810	26.0663

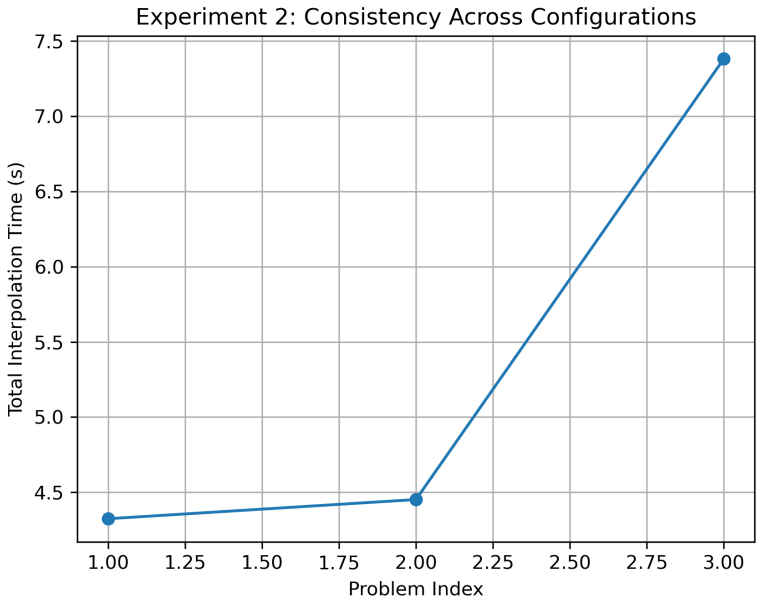

Figure 3: Interpolation consistency across configurations on Lab PC.

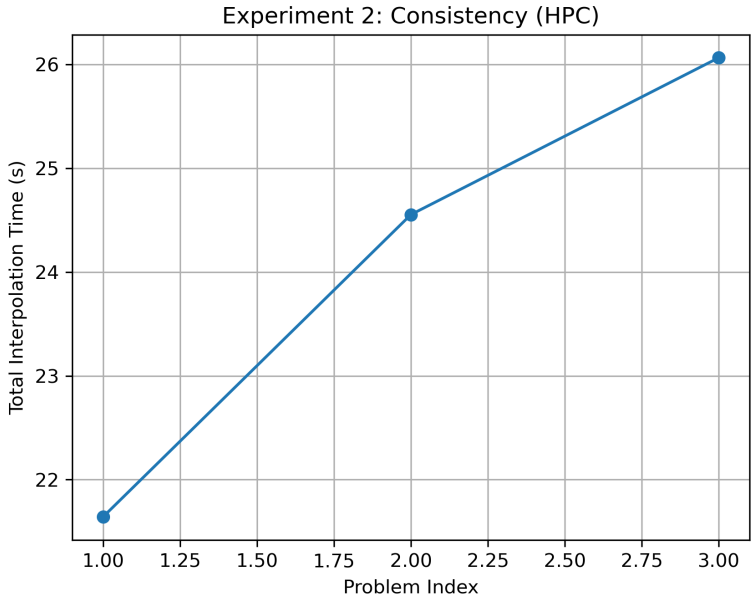

Figure 4: Interpolation consistency across configurations on HPC cluster.

5.3 Experiment 3: Serial vs Parallel Mover

The OpenMP mover significantly reduced runtime on both systems. Average mover times and speedup are shown below.

System	Serial (s)	Parallel (s)	Speedup
Lab PC	0.3337	0.0250	13.33×
HPC	0.2605	0.0533	4.89×

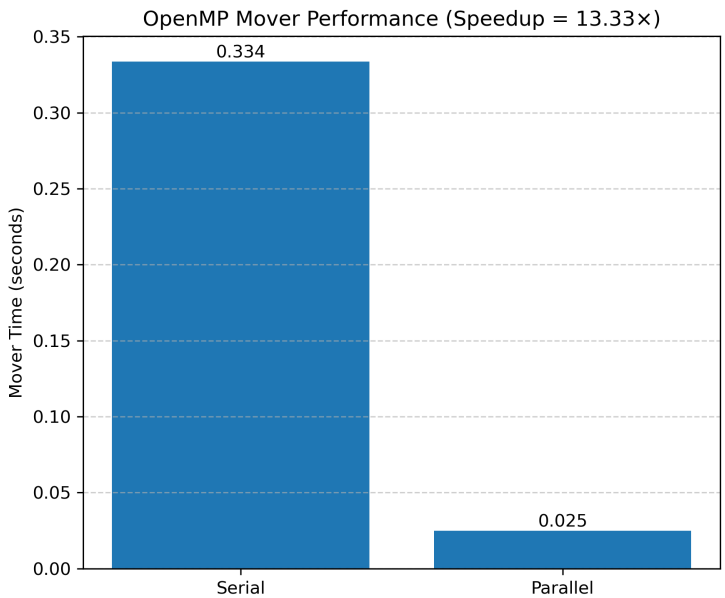

Figure 5: Mover performance comparison on Lab PC.

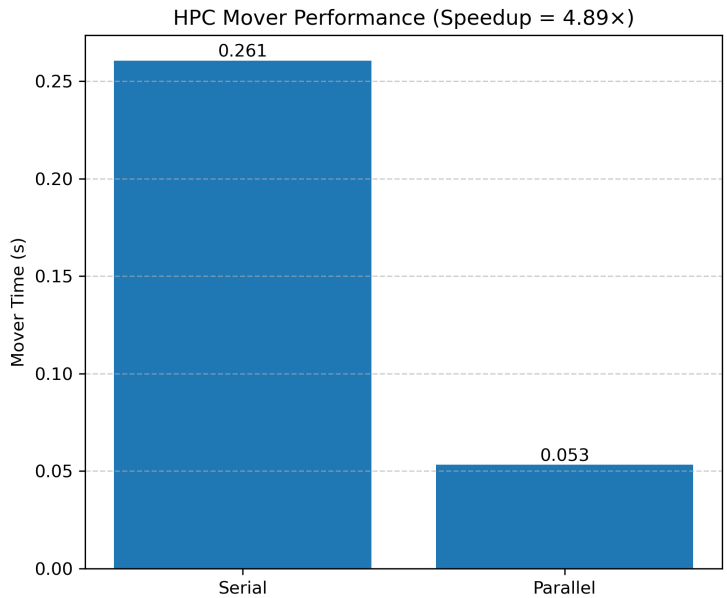

Figure 6: Mover performance comparison on HPC cluster.

5.4 Lab vs HPC Comparison

The Lab PC demonstrated faster interpolation performance for all configurations, while the HPC system showed improved serial mover time due to higher memory bandwidth. However, OpenMP speedup was higher on the Lab PC, indicating better cache locality and thread efficiency on the workstation architecture.

6 Analysis Tasks

Q1. Determine the theoretical time complexity of interpolation and mover operations.

The interpolation algorithm processes each particle independently. For each particle, the algorithm performs constant-time operations: cell index computation, weight evaluation, and four grid updates. Therefore, the interpolation time complexity scales linearly with particle count:

$$T_{\text{interp}}(N) = O(N)$$

Since interpolation is executed for `Maxiter` iterations:

$$T_{\text{interp}} = O(N \cdot \texttt{Maxiter})$$

Similarly, the mover updates each particle independently with a bounded number of random operations and boundary checks. Hence the mover complexity is also:

$$T_{\text{mover}}(N) = O(N \cdot \texttt{Maxiter})$$

Thus, both interpolation and mover scale linearly with particle count.

Q2. Explain why interpolation scaling is nearly linear in Experiment 1.

Experiment 1 results show that interpolation time increases proportionally with particle count across all grid configurations. This occurs because:

- Each particle contributes to exactly four grid nodes.
- The number of arithmetic operations per particle is constant.
- Memory access per particle is fixed (four grid updates).

Therefore, doubling particle count approximately doubles runtime, confirming linear scaling predicted by theory.

Q3. Explain the effect of grid size in Experiment 2.

In Experiment 2, particle count remains fixed while grid size increases. Interpolation time increases moderately with grid resolution due to:

- Larger mesh footprint exceeding cache capacity.
- Reduced temporal locality in grid updates.
- Increased memory latency for scattered writes.

Since particle operations remain constant, grid size affects only memory behavior, not arithmetic workload. Hence the increase in runtime is sublinear compared to particle scaling.

Q4. Explain why the mover achieves high OpenMP speedup.

The mover loop is embarrassingly parallel because each particle update is independent and no shared grid data is modified. Parallel efficiency is high because:

- No synchronization or reduction operations are required.
- Static scheduling evenly distributes particles across threads.
- Each thread uses a private random seed, avoiding contention.

As a result, the parallel mover achieves significant speedup on both systems, reaching 13.33× on the Lab PC and 4.89× on the HPC cluster.

Q5. Explain performance differences between Lab PC and HPC.

Experimental results show:

- Lab PC interpolation faster than HPC.
- HPC serial mover slightly faster.
- Parallel speedup higher on Lab PC.

These differences arise from architectural characteristics:

- Lab PC has higher single-core clock frequency, benefiting serial interpolation.
- HPC nodes have larger memory bandwidth, improving mover memory access.
- HPC cores have lower frequency and deeper memory hierarchy.
- Thread scaling on Lab PC benefits from shared cache locality.

Thus, serial algorithms favor high-frequency desktops, while memory-intensive parallel loops benefit HPC architectures.

Q6. Describe the interpolation–mover workflow pictorially.
The particle–mesh workflow consists of two sequential stages:

1. Interpolation: particle contributions mapped to grid nodes.
2. Mover: particle positions updated stochastically.

Each iteration executes:

$$\text{Particles} \rightarrow \text{Interpolation} \rightarrow \text{Mover} \rightarrow \text{Updated Particles}$$

This pipeline structure highlights that interpolation writes to the mesh, while mover updates particle coordinates. Both operations iterate over particles independently, enabling parallelization of the mover stage without affecting interpolation correctness.

7 Conclusion

- The Cloud-In-Cell (CIC) interpolation algorithm for mapping particle data onto a structured mesh was successfully implemented and validated across multiple grid configurations and particle counts.
- Experimental results confirmed the theoretical time complexity $O(N \cdot \texttt{Maxiter})$, demonstrating near-linear scaling of interpolation runtime with particle count on both Lab PC and HPC systems.
- Interpolation performance showed moderate dependence on grid size due to increased mesh memory footprint and reduced cache locality, indicating that the algorithm is primarily memory-bound.
- The stochastic particle mover was implemented in both serial and OpenMP parallel forms. Parallelization achieved substantial runtime reduction due to the embarrassingly parallel nature of particle updates.
- On the Lab PC, the OpenMP mover achieved a speedup of 13.33×, while the HPC cluster achieved 4.89×, demonstrating effective multicore utilization on both systems.
- Performance comparison revealed that the Lab PC outperformed the HPC cluster for serial interpolation due to higher single-core frequency, whereas the HPC cluster showed improved serial mover performance due to greater memory bandwidth.

- Overall, the study highlights that interpolation is memory-bound and benefits from high clock speed, while the particle mover benefits strongly from parallelization and memory throughput. The results demonstrate the complementary strengths of workstation and HPC architectures for particle–mesh simulations.